

# Test Report

What was tested, what passed, and what the results mean — including two real bugs the tests caught

---

*A plain-language account of the verification: the Java runtime suite, the Python training suite, the label-free separation results, the two latent bugs the tests surfaced and their fixes, and an honest statement of what these results do and do not prove.*

|              |                                                                |
|--------------|----------------------------------------------------------------|
| Document     | Test Report                                                    |
| Product      | Jneopallium Self-Supervised Maintenance Guardian               |
| Java suite   | SelfSupervisedMaintenanceModuleTest — 14/14 passing            |
| Python suite | tests/test_ss_maintenance.py — 5/5 passing                     |
| Method       | Deterministic; fault labels used only to score, never to train |
| Author       | Dmytro Rakovskyi — Kharkiv, Ukraine                            |
| Date         | 2 July 2026                                                    |
| License      | BSD 3-Clause                                                   |

# Contents

---

1. Summary
2. For the non-specialist: how we tested without labels
3. What was under test
4. Java runtime suite (14 tests)
5. Python training suite (5 tests)
6. Label-free separation results
7. Two bugs the tests caught
8. Determinism and reproducibility
9. What these results prove — and what they don't
10. How to reproduce

# 1 Summary

| Suite                                                     | Tests | Result                         |
|-----------------------------------------------------------|-------|--------------------------------|
| Java runtime<br>(SelfSupervisedMaintenanceModule<br>Test) | 14    | all passing                    |
| Python training<br>(test_ss_maintenance)                  | 5     | all passing                    |
| Latent bugs found and fixed                               | 2     | overflow in dedup & rate-limit |

## Headline

Every injected fault family is detected without any label, healthy assets stay quiet, the feedback loop moves thresholds the right way, and the model provably cannot actuate. Two real timestamp-overflow bugs were found by the tests and fixed.

## 2 For the non-specialist: how we tested without labels

A supervised model is easy to test: you hide some labelled failures and check it finds them. This model has no labels, so the test is different. We generate realistic telemetry, secretly inject a slow degradation into some assets, and keep the timing of that degradation in a sealed 'answer key' the model never sees. The model is trained only on healthy data. Then we check: did it flag the degrading assets (using the answer key only to score), and did it leave the healthy ones alone? That is a fair test of a label-free detector.

## 3 What was under test

- **Runtime (Java):** the four neurons, six signals, and five processors — the model that actually runs in production.
- **Training (Python):** the label-free fit and the deployable bundle it emits.
- **Safety invariants:** that the model is advisory and cannot be switched to actuate.

## 4 Java runtime suite (14 tests)

Run through the real JUnit platform. Coverage by area:

| Area           | What is asserted                                                                                                                       |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Reconstruction | healthy frame → low residual; anomaly → high residual on the right sensor; domain-shift flagged                                        |
| Hypothesis     | a rising residual accumulates evidence, elevates severity, attributes the family, and yields a lead time; a healthy stream stays quiet |
| Feedback       | false positive raises the threshold, confirmed relaxes it, adaptation freezes during domain shift, first update is not suppressed      |
| Advisory gate  | gates on threshold, de-duplicates, applies live threshold                                                                              |

|                     |                                                                   |
|---------------------|-------------------------------------------------------------------|
|                     | updates, first advisory not suppressed, advisoryOnly true         |
| Config / processors | advisory-only invariant holds; all processors are interface-typed |

## 5 Python training suite (5 tests)

| Test                           | What it confirms                                                                   |
|--------------------------------|------------------------------------------------------------------------------------|
| baseline percentiles ordered   | mean < p95 < p99 < p999                                                            |
| cross weights finite           | every fitted weight is a finite number                                             |
| healthy frames low residual    | a healthy asset exceeds its own p99 under 5% of the time                           |
| faults separate without labels | each fault family sits above 2× p999 and points at the right sensor group          |
| bundle written and valid       | labelFree=true, labelsUsed=0, safetyMode=ADVISORY, layers carry the fitted weights |

## 6 Label-free separation results

The healthy baseline (fitted on healthy data only): mean 0.78, p99 2.08, p999 2.75 in reconstruction-error units. Against that baseline, every injected degradation — bearing wear, cavitation, sensor drift, energy deterioration, and control oscillation — rises above **twice** the p999 extreme at full ramp, and the dominant residual identifies the correct sensor group in each case. Healthy assets exceed their own p99 less than 5% of the time, and the persistence requirement in the evidence accumulator suppresses most of those transients before they become advisories.

### Why this matters

It shows the label-free core genuinely separates faults from normal using nothing but the coupling between sensors — no failure history was consulted at any point.

## 7 Two bugs the tests caught

The smoke and unit tests surfaced two genuine latent defects, both fixed and now regression-guarded:

- **Advisory-gate dedup overflow.** The per-asset de-duplication used a sentinel of `Long.MIN_VALUE; timestamp - sentinel` overflows for real epoch-millisecond timestamps, which made the check wrongly suppress the **first** advisory for every asset.
- **Feedback rate-limit overflow.** The same sentinel pattern in the adapter would have swallowed the **first** feedback update per family in production.

Both are now a null check instead of a sentinel subtraction, with dedicated tests (`first advisory not suppressed`, `first event is not suppressed`) that use realistic epoch-style timestamps.

## 8 Determinism and reproducibility

---

Training and the synthetic corpus are seeded, so results are bit-for-bit reproducible. The fault timings live only in the scoring oracle; they never enter the fit. Both suites run offline with no third-party Python dependencies.

## 9 What these results prove — and what they don't

---

- **They prove** the runtime logic is correct, the safety invariant holds, and the label-free fit separates the modelled fault families on realistic synthetic data.
- **They do not prove** field accuracy on your plant. Synthetic-corpus separation is not a field detection rate; real assets are messier, and fault families are heuristic until confirmed events accumulate.
- **Therefore:** deploy in shadow first, measure the false-positive rate on your data, and let the feedback loop calibrate — exactly the rollout the Deployment Guide prescribes.

## 10 How to reproduce

---

```
# python training suite
cd scripts/demo-self-supervised-maintenance
python -m unittest tests.test_ss_maintenance -v

# java runtime suite
mvn -pl worker -Dtest=SelfSupervisedMaintenanceModuleTest test
```

*The tests hold the model to an honest bar: detect faults with no labels, never actuate, and improve from feedback — and they earned their keep by catching two real bugs before production.*